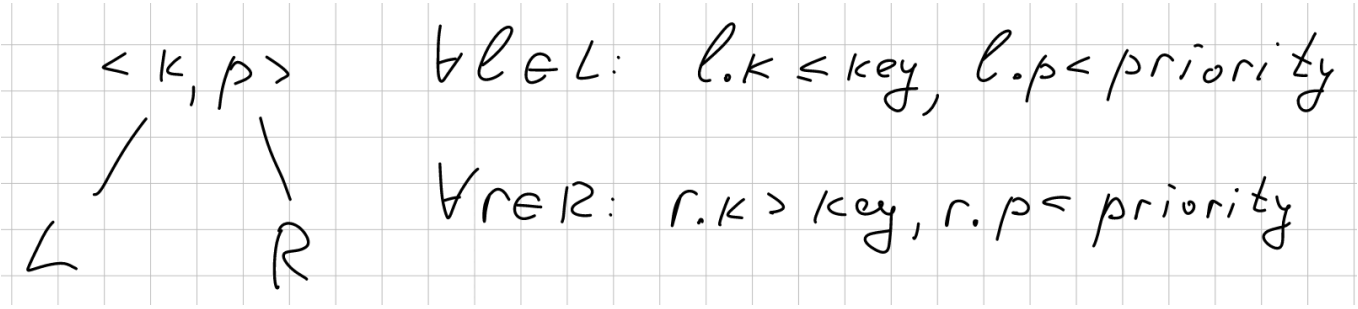
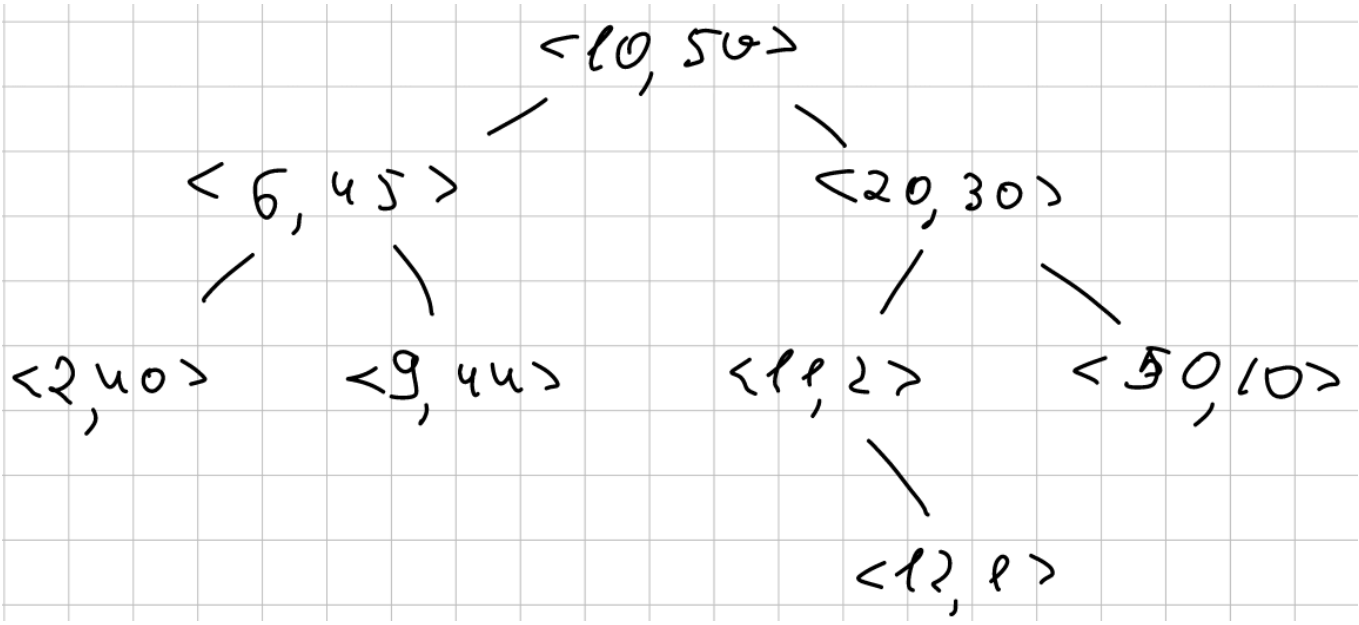
Основные термины

Декартово дерево - структура данных, которая объединяет в себя бинарное дерево (binary tree) и кучу (heap). Узел дерева состоит из пары <ключ, приоритет> (далее <k, p>). Приоритеты должны *равномерно распределяться* специальным генератором! Ключи могут быть любыми и не обязаны задаваться определёнными правилами.

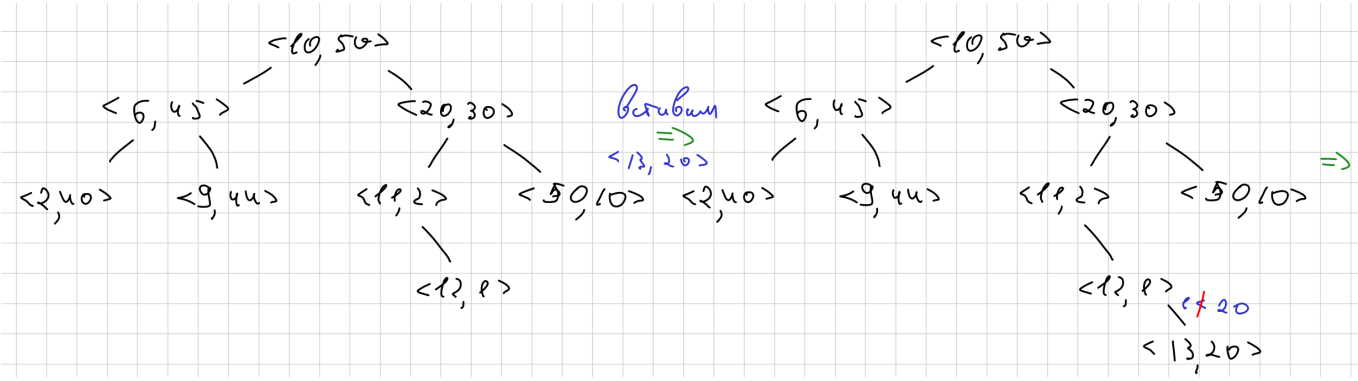


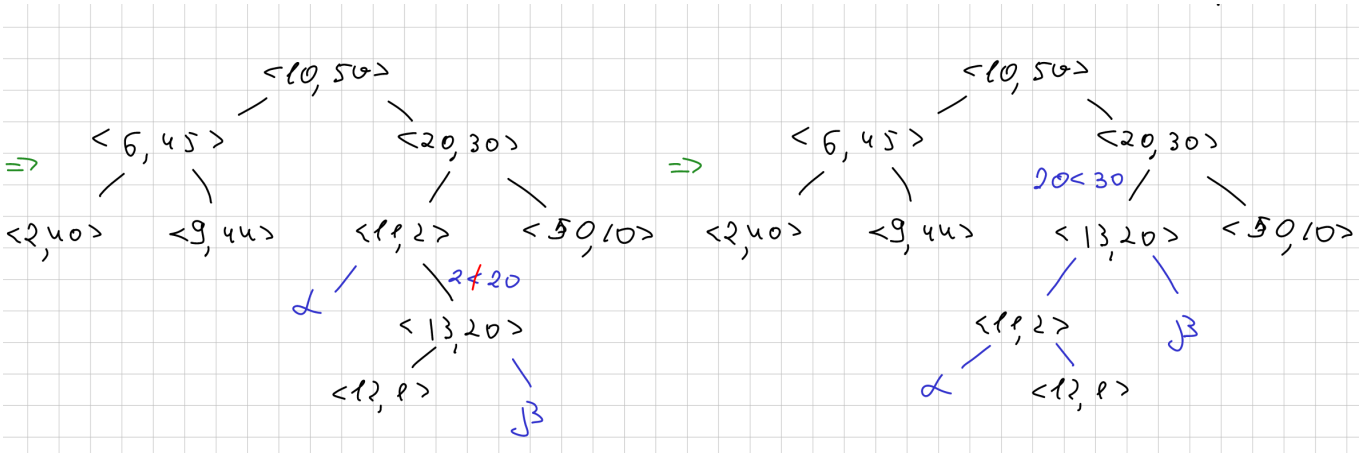
Пример:



Вставка

Поэтапный пример вставки с поворотами:

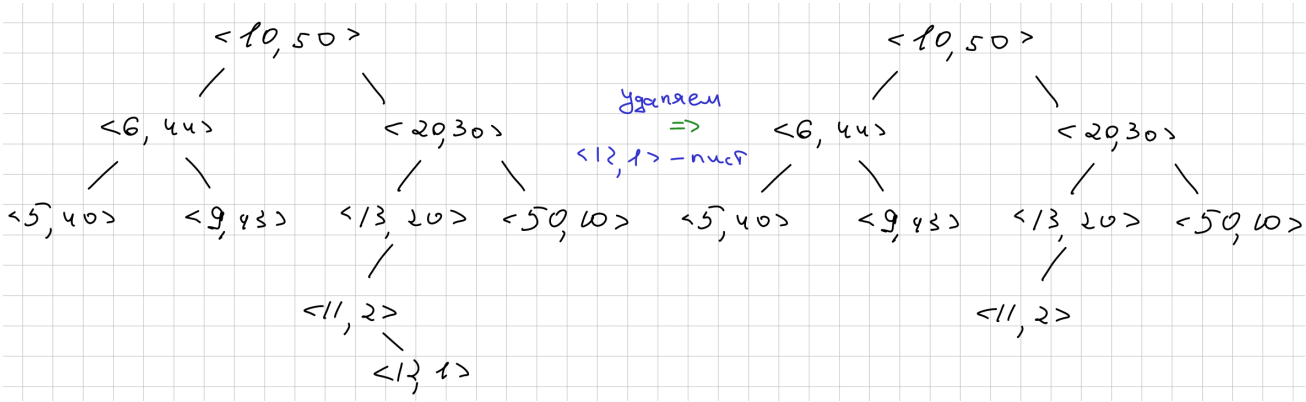




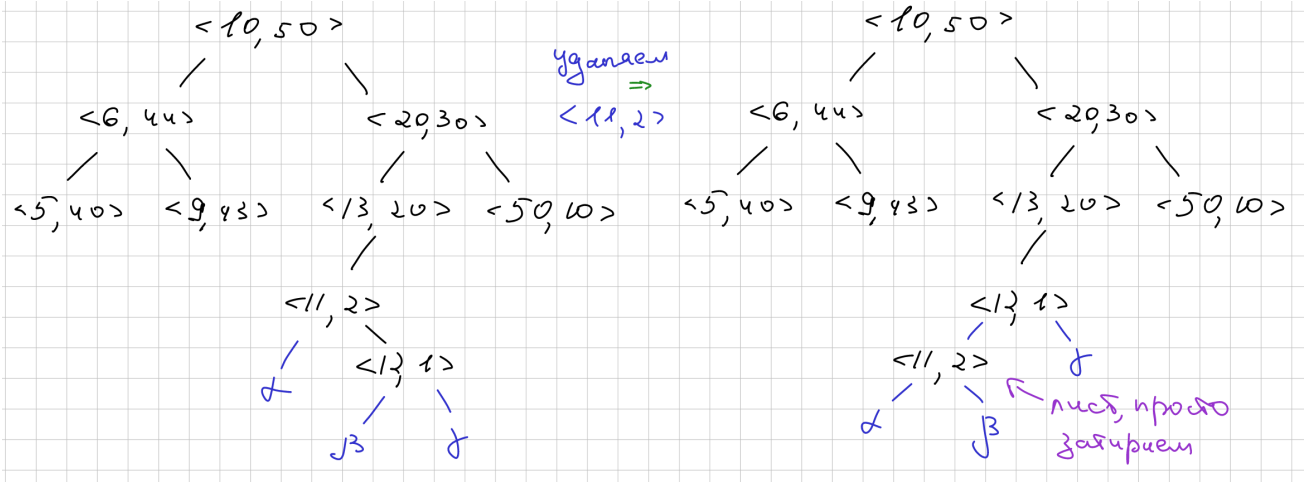
Сложность: $O(h)$ - чтобы найти место, $O(h)$ - проверка правильности приоритета + корректировка. Следовательность $O(h+h) = O(2h) = O(h) \approx O(\log n)$

Удаление

- Если удалеям лист: просто стираем его. Пример:

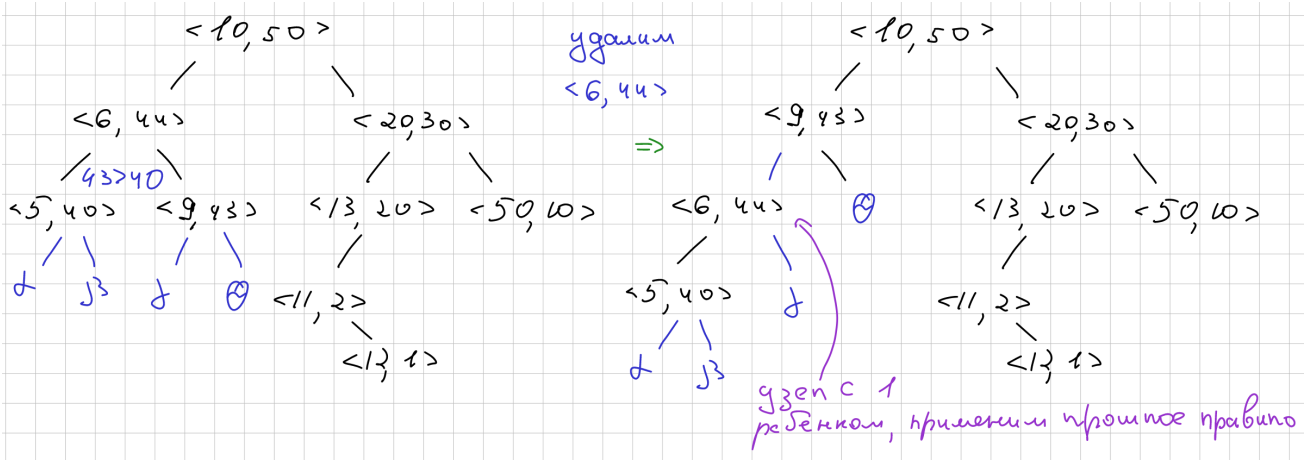


- Если один потомок, то делаем поворот относительно удаялемого узла и потомка. В таком случае удаляемый узел становится листом и мы его просто затираем, сохраняя структуру декартова дерева. Пример:



- Если два потомка: Из двух потомков выбирается тот, у которого выше приоритет. Делаем поворот относительно удаляемого узла и выбранного потомка. В таком случае удаляемый узел становится листом и мы его просто затираем, сохраняя структуру

декартова дерева. Пример:



Сложность: $O(h)$ - чтобы найти, $O(h)$ - чтобы проверить родителей, следовательно, $O(h) \approx O(\log n)$.

SPLIT

Метод `split(T, key)` принимает на ввод исходное дерево `T` и ключ `key`, по которому необходимо делить дерево. Возвращает метод два дерева `T1` и `T2` : $\forall x \in T_1 \quad x.k < key$ и $\forall y \in T_2 \quad y.k > key$

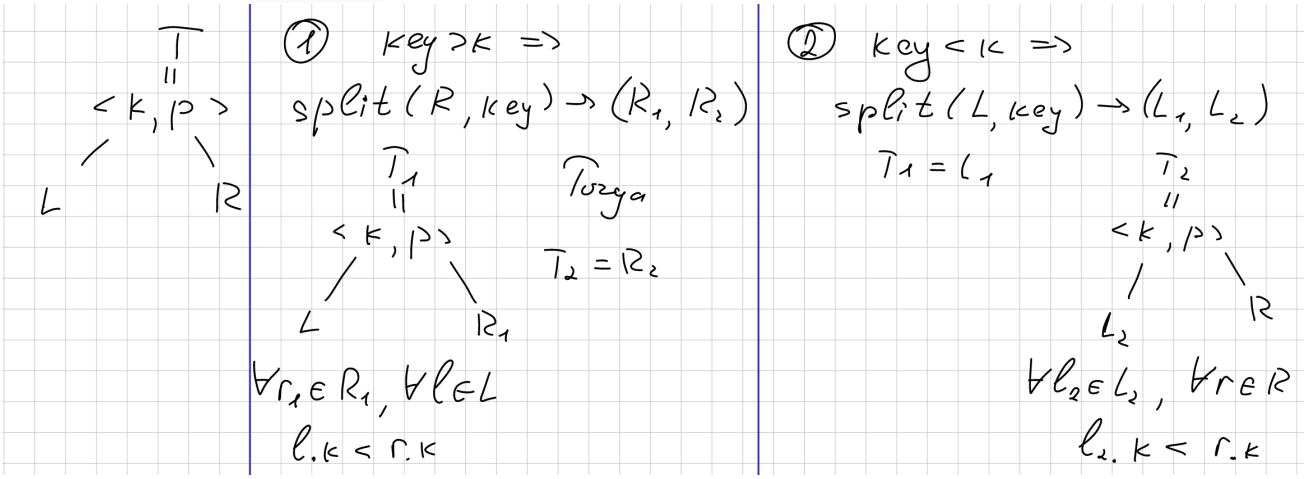
Свойства

- $T_1 \cup T_2 = T$
- $T_1 \cap T2 = \emptyset$

Рассмотрим две ситуации для резки дерева `T = <k, p>`.

1. Допустим `key > k`. Тогда $\forall l \in L, l.k < key$. Тогда в корне `T1` будет лежать узел <k, p>, а его левым ребёнком будет целое поддерево `L`. Для правого поддерева `R` применим `split(R, key)`. Будут получены `R1` и `R2` : $\forall r_1 \in R_1 \quad r_1.k < key$ и $\forall r_2 \in R_2 \quad r_2.k > key$. При этом $\forall r_1 \in R_1$ и $\forall l \in L \quad l.k < r_1.k$. Следовательно, `R1` станет правым потомком дерева `T1` с корнем <k, p>, а `R2 = T2`.
2. Допустим `key < k`. Тогда в корне `T2` будет лежать узел <k, p>, а его правым ребёнком будет целое поддерево `R`. Для левого поддерева `L` применим `split(L, key)`. Будут получены `L1` и `L2` : $\forall l_1 \in L_1 \quad l_1.k < key$ и $\forall l_2 \in L_2 \quad l_2.k > key$. При этом $\forall l_2 \in L_2$ и $\forall r \in R \quad l_2.k < r.k$. Следовательно, `L2` станет левым потомком дерева `T2` с

корнем <k, p>, а `L1 = T1`.



Применение метода `split` завершается как только для выполнения операции поступает пустое дерево. (Мы рекурсивно запускаемся, условие выхода - как только на split пришло пустое дерево)

Сложность: $O(h)$.
Пояснение: необходимо пройти дерево до конца, пока одним из сравниваемых ключей не будет `nullptr`. Худшая ситуация будет тогда, когда это произойдёт на высота $h-1$. Тогда сложность будет $O(h-1) = O(h)$.

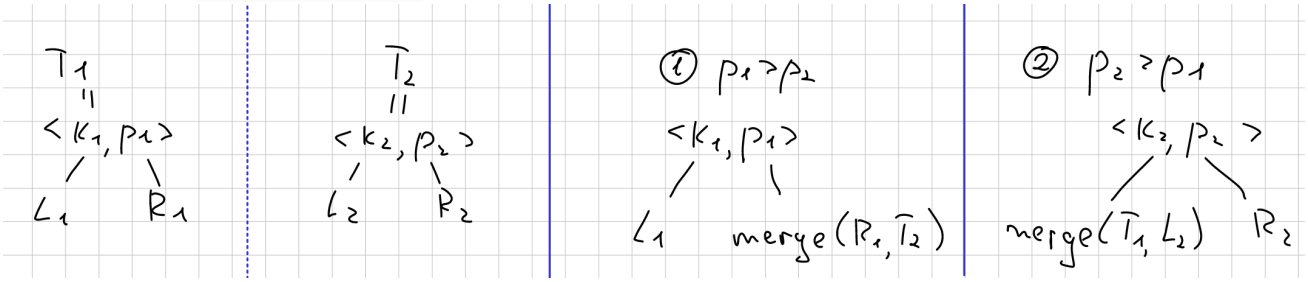
MERGE

Метод `merge(T1, T2)` принимает на ввод два дерева, которое необходимо объединить, чтобы получилось целое дерево `T`. При этом должно выполняться условие $\forall x \in T_1, \forall y \in T_2 \quad x.k < y.k$ (максимальный ключ из `T1` меньше минимального ключа из `T2`).

Пусть нам даны два дерева `T1` и `T2` с корнями `<k1, p1>`, `<k2, p2>`, а также их поддеревья `L1`, `R1`, `L2`, `R2` соответственно. Очевидно, что не один из узлов поддеревьев `L1`, `R1`, `L2`, `R2` не может стать новым корнем в результате применение метода `merge`, т.к. приоритеты любого узла поддеревьев дерева `T1` меньше `p1`, аналогично для `T2`. Следовательно, корнем применения `merge` станет либо `<k1, p1>`, либо `<k2, p2>`. Для этого нужно рассмотреть две ситуации:

1. $p_1 > p_2$. Тогда `<k1, p1>` будет корнем итогово дерева. `L1` останется левым потомком, т.к. $\forall l_1 \in L_1, \forall t \in T_2 \quad l_1.k < t.k$. Правым же потомком станет результат выполнения `merge(R1, T2)`.
2. $p_1 < p_2$. Тогда `<k2, p2>` будет корнем итогово дерева. `R2` останется правым потомком, т.к. $\forall r_2 \in R_2, \forall t \in T_1 \quad t.k < r_2.k$. Левым же потомком станет результат

выполнения `merge(T1, L2)`.



Рекурсивные вызовы `merge` завершаются сразу после обработки ситуации, когда одно пришедшее дерево - `nullptr`, а второе существует. Именно его и нужно вернуть, т.к. бессмысленно сравнивать с `nullptr`.

Сложность: $O(h_1 + h_2)$, где $h(T_1) = h_1$ и $h(T_2) = h_2$.
Пояснение: В любом случае операции `merge` будет рекурсивно продолжаться, пока одно из деревьев не станет пустым. Это ситуация возможна с обеими операциями. Тогда худшим вариантом является ситуация, когда будет чередоваться результат сравнения `pr1` и `pr2`. В таком случае будет сделано $h_1 + h_2$ вызов `merge`. Следовательно, в худшем случае будет $O(h_1 + h_2)$.

Вставка с split/merge

```
insert(T, <key, pr>):
     split(T, key) -> (T1, T2)
     createTree(<key, pr>) -> T_new
     merge(T1, T_new) -> T1'
     merge(T1', T2) -> T'
     return T'
```

Пояснение: В `T1` получилось дерево, ключи которого меньше вставляемого ключа, в `T2` - больше. Создаём дополнительное дерево из одного элемента и объединяем его с `T1`. Далее объединяем получившееся дерево с `T2`. Результат последей операции будет исходное дерево со вставленным в него `<key, pr>`.

Удаление с split/merge

```
remove(T, <key, pr>):
     split(T, key) -> (T1, T2)
     split(T1, key-eps) -> (T1', T2') # eps - очень малое значение
     merge(T1', T2) -> T_new
```

1. $\forall t_1 \in T_1 \quad t_1.k \leq key$
   $\forall t_2 \in T_2 \quad t_2.k > key$

2. $\forall t'_1 \in T'_1 \quad t'_1.k < key$
   $\forall t'_2 \in T'_2 \quad t'_2.k = key$
3. $\forall t_{ne} \in T_{ne} \quad t_{ne}.k \neq key$

Неявные декартовы деревья

> Будут потом